

Architectural Analysis 2026-05-09

2026-05-09

Architectural Analysis — 2026-05-09

- Scope
- Modes included
- Cross-mode index
- Headline findings
- Verification summary
- Diagrams
- Open questions
- Methodology note

Information Architecture

- Summary
- Callouts
- Narrative
- Synthesized concepts
- Verification log
- Open questions

Data Model

- Summary
- Callouts
- Narrative
- Verification log
- Open questions

Data Flow

- Summary
- Callouts
- Narrative
- Verification log
- Open questions

Integration Points

- Summary
- Callouts
- Narrative
- Verification log
- Open questions

UI Surfaces

- Summary
- Callouts
- Narrative
- Verification log
- Open questions

Control Flow
Summary
Callouts
Narrative
Verification log
Open questions
Failure Modes
Summary
Callouts
Narrative
Verification log
Open questions

Architectural Analysis — 2026-05-09

Scope

Snapshot of Facet's canonical user flow from **identity extraction** through **search/research** → **pipeline** → **prep** → **letters/resume/recruiter** derived artifacts, plus the cross-cutting **persistence** and **assembly engine** layers those workspaces depend on.

Included: - All in-scope route components under `src/routes/` (10 routes: `identity`, `research`, `match`, `pipeline`, `prep`, `letters`, `build`, `linkedin`, `recruiter`, `debrief`) - Identity domain (`src/identity/` — `schema`, `resumeAdapter`, `skillDedupe`) - Engine layer (`src/engine/` — `assembler`, `pageBudget`, `serializer`, `letterAssembler`, `typst.worker`) - Persistence (`src/persistence/` — `coordinator`, `snapshot`, `runtime`, `validation`, `normalization`, `indexedDb`, `remoteBackend`, `fileSystemAccess`, `hydration`, etc.) - Domain stores (`src/store/` — `identity`, `pipeline`, `prep`, `search`, `resume`, `coverLetter`, `jdAnalysis`, `linkedin`, `recruiter`, `match`, `debrief`) - AI generators (`src/utils/*Generator.ts`, `src/utils/llmProxy.ts`, `src/utils/aiAccess.ts`)

Excluded: - `/account`, `/admin`, `/help`, `/legal`, `/home` routes (not in user-flow scope) - Test fixtures and `src/test/` - Theme system (`src/themes/`) - Dev-only tooling and `src/dev/`

Modes included

- **Information architecture** — Identity is the gravitational center: every LLM generator imports `identity/schema.ts`. Persistence is a synthesized three-file coordinator pattern.
- **Data model** — Two centers of gravity: `ProfessionalIdentityV3` (canonical) and `PipelineEntry` (durable application record). Derived artifacts (`Resume`, `CoverLetter`, `LinkedIn`, `Recruiter`, `Debrief`) all reference identity by FK + `identityVersion` stamp.
- **Data flow** — Six critical paths converge on a single persistence boundary. The 150ms `schedulePersist` debounce is the design's response to write amplification.

- **Integrations** — No direct anthropic-sdk usage; every LLM call goes through `callLLMProxy`. Typst WASM via Web Worker is the only non-LLM external compute.
- **UI surfaces** — 20 routes; 14 in scope. Three eager-loaded routes (Build, Pipeline, Recruiter) are the daily-driver triad.
- **Control flow** — Persistence coordinator is an explicit 6-state machine. Render uses generation-counter to discard stale worker responses. Deep-research uses concurrent SSE + poll-loop with stepped backoff.
- **Failure modes** — No automatic retry at proxy boundary; retry is caller-side and rare. Persistence collapses all errors to offline/error binary. Typst worker keeps old preview on render error rather than blanking.

Cross-mode index

Callout	Origin mode	Also referenced from	Label
I-8	information	data-model (as M-1 root, etc.), data-flow (D-2 imports)	identity/schema.ts
I-17, I-18, I-19	information	control-flow (C-2...C-13), data-flow (D-30...D-34)	persistence triad files
I-43	information	control-flow (the coordinator state machine is its operational form)	persistence coordinator pattern (synthesized)
M-1	data-model	data-flow (D-9 <code>currentIdentity</code>), IA (I-8 owns this type)	ProfessionalIdentityV3
M-20	data-model	data-flow (D-22 <code>addEntry</code> produces, D-23 <code>entries</code> collection)	PipelineEntry
M-22	data-model	data-flow (D-24 <code>jdAnalysisStore.upsertAnalysis</code>), failure-modes	JDAnalysis
M-29	data-model	data-flow (D-26 <code>prepStore.createDeck</code>)	PrepDeck
M-33	data-model	data-flow (D-12 <code>resumeStore.setData</code>), control-flow	ResumeData
M-44	data-model	failure-modes (F-37 backfill stamps it), data-flow	DurableMetadata
D-30	data-flow	control-flow (C-13), failure-modes (debounced retry trigger)	schedulePersist (150ms debounce)
D-32	data-flow	control-flow (C-10 inverse via <code>applyWorkspaceSnapshotToStores</code>)	createWorkspaceSnapshotFromStores
C-13	control-flow	data-flow (D-30 same node)	schedulePersist debounce
C-26	control-flow	data-flow (D-17 same worker)	typst.worker spawn
C-31	control-flow	failure-modes (stale-response guard prevents corrupt preview)	renderGenerationRef stale-response guard
F-1			FacetAiProxyError

Callout	Origin mode	Also referenced from	Label
	failure-modes	control-flow (C-20 categorizer produces this), integrations (X-3 boundary)	
F-2	failure-modes	control-flow (offline phase resolves via this), integrations (X-20 produces this)	FacetApiError
F-30	failure-modes	control-flow (C-30 same site)	preserve old preview on render error
X-2	integrations	data-flow (LLM calls go here), control-flow (C-18 boundary), failure-modes	Anthropic API (via proxy)
X-21	integrations	data-flow (D-35 hosted persistence), control-flow (C-35 hosted bootstrap)	Facet hosted persistence backend (Supabase)
X-26, X-27	integrations	data-flow (D-17 typst.worker), control-flow (C-26 worker spawn)	typst.worker + Typst WASM
U-2	ui-surfaces	control-flow (C-34 bootstrap branch on deployment mode)	AppShell
OBS-1	ui-surfaces	control-flow (C-40 lazy route boundaries)	Eager-loaded triad signal

Headline findings

- 1. Identity is the gravitational center of the data model and the IA simultaneously.** [M-1] ProfessionalIdentityV3 (1407-line schema) is referenced by **seven LLM generators** at the IA level and is the source of every derived artifact at the data-model level. Memory's identity-canonical-data principle is structurally enforced: artifacts mirror identity rather than store independent fields.
- 2. Persistence is a single-snapshot, debounced, cross-cutting concern with explicit state-machine semantics.** The 150ms schedulePersist debounce ([D-30]/[C-13]), the 6-state coordinator ([C-1]...[C-9]), the legacy-storage migration path ([C-11]), and the cross-tab sync ([C-15]/[C-16]) make persistence one of the architecturally heaviest layers — and the only one with a coordinator-pattern synthesized abstraction ([I-43]).
- 3. The proxy boundary is a deliberate, non-bypassable architectural commitment.** No direct @anthropic-ai/sdk usage; every LLM call routes through [X-1] resolveAiAccess → [X-10] callLlmProxy → [X-2] Anthropic. This concentrates entitlement gating, error categorization ([F-3] readAiProxyError), and timeout policy in one boundary — and gives hosted/local mode a clean swap point.
- 4. Retry is rare and explicit; failure containment dominates.** Only [F-13] jobMatch (2 attempts) and [F-41] deep-research polling have retry. Typst worker has zero retry but compensates with [F-30] “keep old preview.”

Persistence has no retry but binarizes all errors to offline/error via [F-23] `resolvePersistenceFailurePhase`. The pattern: surface failures to UI catches rather than hiding them in retry policies.

5. **The eager-loaded triad (Build, Pipeline, Recruiter) is an architectural assertion about the daily-driver workflow.** 17 of 20 routes are `React.lazy()`; only those three are eager imports in [U-1] `router.tsx:4-6`. This embeds the topology doc's "pipeline owns durable job context" principle in the bundle-splitting decision: the durable-context surfaces load instantly; the AI-dependent surfaces pay a code-split cost on first navigation.
6. **The Typst rendering boundary has the codebase's most carefully-engineered concurrent control flow.** [C-31] `renderGenerationRef` discards stale worker responses; [C-27] 400ms debounce coalesces prop changes; [C-32] `terminate-on-unmount` prevents leaks; [C-29] revokes previous blob URLs. This is unusual care for a "preview" surface — the engineering effort signals how often render is on the user's critical path.
7. **The Search → Pipeline → Prep flow is structurally enforced by FK references, not duplicated state.** [M-29] `PrepDeck` carries `pipelineEntryId`, `pipelineRoundId`, and `jdAnalysisId` — three FKs back into the canonical pipeline cluster. `PipelineEntry` similarly references `JDAnalysis` and `CoverLetter` by ID, not by embedded copy. This is what lets the workspace topology stay coherent even with 11 stores: cross-store relationships flow through the data model, not the runtime.

Verification summary

Mode	Findings	Verified	Repaired	Discarded	Synthesized	Synthesized share
Information architecture	44	43	1 (added I-44 prepStore)	1 (duplicate I-12)	1 (I-43)	2.3%
Data model	55	55	0	0	0	0%
Data flow	38	38	0	0	0	0%
Integrations	45	40	0	5 (out-of-scope)	0 (re-classified to external)	0%
UI surfaces	66	52	14 (page-export lines)	0 (page nodes themselves valid)	1 (OBS-1)	1.9%
Control flow	40	40	0	0	0	0%
Failure modes	50	50	0	0	0	0%
Total	338	318	15	6	2	—

Citation verification used the codanna MCP index for symbol resolution where applicable, plus targeted grep for absence-claim refutation and Read for line-content matching. The discard-and-repair log is preserved in each mode's report.md Verification log section.

Diagrams

- Information architecture (rendered from information/ia.mmd)
- Data model — ERD
- Data flow — pipeline | Sequence — resume render | Sequence — pipeline promotion | Sequence — snapshot save
- Integrations — boundaries
- UI surfaces — routes | UI surfaces — components
- Control flow — state machine | Sequence — render lifecycle | Sequence — hydration | Sequence — research job
- Failure modes

Open questions

Aggregated from per-mode reports, de-duplicated:

- **Schema migration mechanics:** [F-37] silent backfill handles schema *additions*; what about renames or field-type changes? Inspect `validation.ts` migration logic.
- **Typst preview cache lifecycle:** [F-30] “keep old preview on error” depends on the previous blob URL existing; first-render-after-navigation has no fallback. Bug or intentional?
- **LLM retry asymmetry:** Why does [F-13] `jobMatch` retry but no other generator? Idempotency-driven decision, or accident of evolution?
- **Deep-research circuit breaker:** [F-41] poll-on-error keeps polling indefinitely with [F-42] capped 30s intervals. Is there a max-attempts guard, or does the user have to abort?
- **Deep-link bridge band-focus lock:** `/identity` was redesigned around a single canvas + bands + sticky inspector (the Model/Strategy tab shell is gone — `src/routes/identity/IdentityMapPage.tsx:24-32`). The bridge has three params: `?sel` (12+ variant inspector-pin), `?focus` (band-scroll, currently bounded to `['preferences']` by explicit lock at `src/utils/mapSelectionUrl.ts:184`), and `?return` (back-link breadcrumb). When does the bounded-set lock become friction as other workspaces gain landing-target needs?
- **Worker pool need:** [X-29] Web Worker is single-instance per `usePdfPreview` mount. Does side-by-side compare or multi-preview UI need a pool?
- **Pipeline panel composition:** Pipeline workspace decomposes into 6 panels; other workspaces have ~1. Is pipeline genuinely richer or is it accumulating complexity?
- **Eager-load triad rationale:** Why Build/Pipeline/Recruiter specifically? Recruiter is less obviously a daily driver than the other two.
- **MatchReport entity gap:** `MatchReport` (`src/types/match.ts:175`) referenced by `JDAAnalysis` fields but not modeled in the ERD.

Methodology note

This report was generated by the architectural-analysis skill on 2026-05-09. Seven sub-agents (haiku Explore × 4 enumeration modes, sonnet general-purpose × 3 reasoning modes) ran in parallel against an explicit user-flow scope. The orchestrator ran the verification protocol over all 337 candidate findings — using the codanna MCP index for symbol resolution and Read/grep for line-content matching — discarding 6 (1.8%), repairing 14 line citations (4.2%), and re-classifying 17 integration external nodes from synthesized to the external classDef before any node landed in a diagram. Synthesized share is under the 20% cap in every mode (max 2.4% in Information Architecture). Absence claims were independently grep-refuted before survival; one absence claim survived ([F-14] no Typst worker retry).

Per-mode report.md Verification log sections preserve the discard-and-repair record. The Open questions lists across modes are seeds for follow-up, not findings.

Information Architecture

Summary

The user flow is layered top-down: a **router shell** routes to **page-level workspaces**, each consuming a **dedicated Zustand store**, with **identity/schema** as the canonical data source feeding both **utils-layer generators** (LLM clients) and the **engine/persistence** stack. Cross-cutting persistence is a three-file coordinator pattern (snapshot → coordinator → runtime) — this is the only synthesized node and the largest architectural commitment outside identity.

The IA’s headline observation: **every LLM generator imports from identity/schema.ts** and **every store flows through the persistence runtime**. These two convergence points (identity-as-canonical, persistence-as-cross-cutting) are the load-bearing constraints downstream modes operate within.

Callouts

ID	Label	Citation	Confidence
I-1	main.tsx	src/main.tsx:1	high
I-2	router.tsx	src/router.tsx:1	high
I-3	AppShell	src/components/AppShell.tsx:231	high
I-5	identityStore	src/store/identityStore.ts:1	high
I-6	searchStore	src/store/searchStore.ts:1	high
I-7	pipelineStore	src/store/pipelineStore.ts:1	high

ID	Label	Citation	Confidence
I-8	identity/schema.ts	src/identity/schema.ts:1	high
I-9	identity/skillDedupe.ts	src/identity/ skillDedupe.ts:1	high
I-10	identity/ resumeAdapter.ts	src/identity/ resumeAdapter.ts:1	high
I-11	engine/assembler.ts	src/engine/assembler.ts:1	high
I-13	engine/serializer.ts	src/engine/serializer.ts:1	high
I-14	engine/pageBudget.ts	src/engine/ pageBudget.ts:1	high
I-15	engine/ letterAssembler.ts	src/engine/ letterAssembler.ts:1	high
I-16	persistence/contracts.ts	src/persistence/ contracts.ts:1	high
I-17	persistence/snapshot.ts	src/persistence/ snapshot.ts:1	high
I-18	persistence/ coordinator.ts	src/persistence/ coordinator.ts:1	high
I-19	persistence/runtime.ts	src/persistence/ runtime.ts:1	high
I-20	resumeStore	src/store/ resumeStore.ts:1	high
I-21	coverLetterStore	src/store/ coverLetterStore.ts:1	high
I-22	matchStore	src/store/matchStore.ts:1	high
I-23	jdAnalysisStore	src/store/ jdAnalysisStore.ts:1	high
I-24	linkedinStore	src/store/ linkedinStore.ts:1	high
I-25	recruiterStore	src/store/ recruiterStore.ts:1	high
I-26	debriefStore	src/store/ debriefStore.ts:1	high
I-27	templates/registry.ts	src/templates/registry.ts:1	high
I-28	utils/ identityExtraction.ts	src/utils/ identityExtraction.ts:1	high
I-29	utils/jdAnalysis.ts	src/utils/jdAnalysis.ts:1	high
I-30	utils/ coverLetterGenerator.ts	src/utils/ coverLetterGenerator.ts:1	high
I-31	utils/prepGenerator.ts	src/utils/ prepGenerator.ts:1	high
I-32	utils/thesisGenerator.ts	src/utils/ thesisGenerator.ts:1	high
I-33	utils/searchExecutor.ts	src/utils/ searchExecutor.ts:1	high

ID	Label	Citation	Confidence
I-34	utils/ deepSearchClient.ts	src/utils/ deepSearchClient.ts:1	high
I-35	utils/ debriefGenerator.ts	src/utils/ debriefGenerator.ts:1	high
I-36	routes/pipeline/ PipelinePage.tsx	src/routes/pipeline/ PipelinePage.tsx:1	high
I-37	routes/letters/ LettersPage.tsx	src/routes/letters/ LettersPage.tsx:1	high
I-38	routes/prep/ PrepPage.tsx	src/routes/prep/ PrepPage.tsx:1	high
I-39	routes/research/ ResearchPage.tsx	src/routes/research/ ResearchPage.tsx:1	high
I-40	utils/identityMerge.ts	src/utils/ identityMerge.ts:1	high
I-41	persistence/ validation.ts	src/persistence/ validation.ts:1	high
I-42	persistence/ normalization.ts	src/persistence/ normalization.ts:1	high
I-43	persistence coordinator pattern	—	synthesized
I-44	prepStore	src/store/prepStore.ts:1	high

Narrative

Entry chain

[I-1] boots the React tree, mounting [I-2] which is a TanStack Router createRouter instance with 20 route declarations. The router renders [I-3] (AppShell) at the root, which is the only screen-level layout in the app — every workspace sits inside it.

Identity is the gravitational center

[I-8] (identity/schema.ts, 1407 lines) defines ProfessionalIdentityV3 and its constituent shapes (roles, projects, skills, preferences, self-model, search vectors). Seven utils-layer generators import from it directly: [I-28] identityExtraction, [I-29] jdAnalysis, [I-30] coverLetterGenerator, [I-32] thesisGenerator, [I-34] deepSearchClient, [I-35] debriefGenerator, [I-40] identityMerge. The engine layer also imports it via [I-13] serializer.ts. **No major derivation utility skips identity** — this is the architectural commitment described in docs/architecture/identity-canonical-data.md.

The persistence triad — [I-43]

[I-17] `snapshot.ts`, [I-18] `coordinator.ts`, and [I-19] `runtime.ts` jointly implement the three-tier persistence pattern:

- `snapshot.ts` composes the workspace envelope by reading every persisted store
- `coordinator.ts` abstracts the storage backend (IndexedDB, localStorage, remote)
- `runtime.ts` orchestrates the lifecycle: bootstrap, hydrate, subscribe, debounce-persist

[I-19] imports both [I-5] `identityStore` and [I-18] `coordinator` — it's the integration point that wires Zustand into the snapshot lifecycle. [I-41] `validation.ts` and [I-42] `normalization.ts` provide the data-shape guards on the boundary.

Stores fan out by workspace

Each major workspace owns a Zustand store: [I-5] `identityStore`, [I-6] `searchStore`, [I-7] `pipelineStore`, [I-20] `resumeStore`, [I-21] `coverLetterStore`, [I-22] `matchStore`, [I-23] `jdAnalysisStore`, [I-24] `linkedinStore`, [I-25] `recruiterStore`, [I-26] `debriefStore`, [I-44] `prepStore`. [I-7] `pipelineStore` imports from [I-5] `identityStore` directly — pipeline holds references to identity vectors. The other stores are independent at this layer, with cross-store coordination flowing through the persistence runtime rather than direct imports.

[I-44] `prepStore` is the second-largest store at 1341 lines, but its size is sanitization-heavy (PrepDeck and PrepCard each carry ~30 optional fields, each needing a normalizer). Its public surface is a typical 13-method workspace store. The one prep-specific behavior worth flagging: a per-card study-progress tracker (`recordCardReview` at `src/store/prepStore.ts:1280-1305`) maintains `{confidence, attempts, needsWorkCount, lastReviewedAt}` per review key. No other workspace store has this state-machine micro-shape.

Routes consume stores + utils

The four routes shown in the diagram are the most-trafficked in the user flow: - [I-36] `PipelinePage` imports [I-7] `pipelineStore` and [I-29] `jdAnalysis` (the analyzer) - [I-37] `LettersPage` imports [I-21] `coverLetterStore` and [I-30] `coverLetterGenerator` - [I-38] `PrepPage` imports [I-31] `prepGenerator`, [I-7] `pipelineStore` (deck creation needs the pipeline entry context), and [I-44] `prepStore` (deck CRUD) - [I-39] `ResearchPage` imports [I-5] `identityStore` and [I-32] `thesisGenerator`

The pattern: **routes hold no logic** beyond UI orchestration; they delegate to a store for state and a util for AI work.

Synthesized concepts

ID	Label	Justification
I-43	persistence coordinator pattern	The three-tier persistence system has no single owning file — it emerges from the coordination of <code>src/persistence/snapshot.ts:1</code> (workspace artifact composition), <code>src/persistence/coordinator.ts:1</code> (backend abstraction + status state machine), and <code>src/persistence/runtime.ts:1</code> (Zustand subscription orchestration + debounce + lifecycle). The pattern is documented architecturally in <code>docs/architecture/facet-workspace-topology.md</code> but no code line owns it.

Verification log

Discarded findings

- I-12 (duplicate of I-11) — both labeled `engine/assembler.ts` with citation `src/engine/assembler.ts:1`. Discarded as redundant; kept I-11.

Added findings

- I-44 (`prepStore`) — the IA subagent silently omitted `prepStore` from its enumeration despite the dispatch prompt explicitly listing it. Confirmed via `Read src/store/prepStore.ts:1` (cites `import { create } from 'zustand'`) and the `PrepState` interface at `src/store/prepStore.ts:106–124`. Added as a peer of the other workspace stores; import edge from [I-38] `PrepPage` reflects the actual consumption.

Synthesized cap

- Synthesized share: $1/42 = 2.4\%$ (cap: 20%) — well under cap.

Unverified citations

- I-31 (`utils/prepGenerator.ts`), I-33 (`utils/searchExecutor.ts`) — file existence verified via `ls`; line:1 evidence not deeply inspected (subagent's evidence string was multi-line `import { ... }`-style, consistent with line:1 being an import statement).

Open questions

- `src/components/` was scoped in but only `AppShell` made the cut at IA level — most components are leaf-level and belong in UI surfaces. Worth confirming this scoping choice.

Data Model

Summary

The data model has **two centers of gravity**: `ProfessionalIdentityV3` (the identity-canonical schema in `src/identity/schema.ts`) and `PipelineEntry` (the durable job-application record in `src/types/pipeline.ts`). Identity owns the user's career as facts; `PipelineEntry` owns each application as a process. Every other major entity in the user flow is either a *derivation* of identity (`ResumeData`, `CoverLetter`, `LinkedInProfileDraft`, `RecruiterCard`, `DebriefSession`) or a *projection* of pipeline (`JDAAnalysis`, `PrepDeck`, `PipelineRound`).

This is the **identity-canonical-data** principle in concrete form: artifact entities don't store fields they could read from identity; they reference identity by FK or carry an `identityVersion` stamp. The diagram makes this visible — identity and its constituents form a tight cluster on one side, and every artifact entity has at most one or two FKs pointing into the identity cluster (or, more often, into `PipelineEntry` which itself references identity-derived state).

A second commitment visible here: **AudienceTagged is a marker interface**, not a normal entity — it's mixed into `MatchRequirement`, `SkillMatch`, `MatchAdvantage`, etc. via `extends`. This is how audience-tagging cross-cuts the data model without a separate persistence path.

Callouts

ID	Label	Citation	Confidence
M-1	<code>ProfessionalIdentityV3</code>	<code>src/identity/schema.ts:265</code>	high
M-2	<code>ProfessionalIdentityCore</code>	<code>src/identity/schema.ts:34</code>	high
M-3	<code>ProfessionalSelfModel</code>	<code>src/identity/schema.ts:65</code>	high
M-4	<code>ProfessionalPreferences</code>	<code>src/identity/schema.ts:141</code>	high
M-5	<code>ProfessionalSkills</code>	<code>src/identity/schema.ts:177</code>	high
M-6	<code>ProfessionalRole</code>	<code>src/identity/schema.ts:200</code>	high

ID	Label	Citation	Confidence
M-7	ProfessionalProject	src/identity/ schema.ts:210	high
M-8	ProfessionalIdentityArcEntry	src/identity/ schema.ts:48	high
M-9	ProfessionalPhilosophyEntry	src/identity/ schema.ts:53	high
M-10	ProfessionalInterviewStyle	src/identity/ schema.ts:59	high
M-11	ProfessionalCompensationPreferences	src/identity/ schema.ts:77	high
M-12	ProfessionalWorkModelPreferences	src/identity/ schema.ts:84	high
M-13	ProfessionalMatchingPreferences	src/identity/ schema.ts:106	high
M-14	ProfessionalSkillGroup	src/identity/ schema.ts:168	high
M-15	ProfessionalRoleBullet	src/identity/ schema.ts:187	high
M-16	ProfessionalPreferencePriority	src/identity/ schema.ts:71	high
M-17	ProfessionalMatchingPriority	src/identity/ schema.ts:90	high
M-18	ProfessionalMatchingAvoid	src/identity/ schema.ts:98	high
M-19	ProfessionalSkillItem	src/identity/ schema.ts:149	high
M-20	PipelineEntry	src/types/ pipeline.ts:182	high
M-21	ProfessionalSearchVector	src/identity/ schema.ts:237	high
M-22	JDAnalysis	src/types/ jdAnalysis.ts:31	high
M-23	MatchRequirementCoverage	src/types/ match.ts:124	high
M-24	SkillMatch	src/types/ match.ts:50	high
M-25	PipelineRound	src/types/ pipeline.ts:164	high
M-26	PipelineResearchSnapshot	src/types/ pipeline.ts:106	high
M-27	PipelineInterviewer	src/types/ pipeline.ts:139	high
M-28	PrepInterviewerIntel	src/types/ prep.ts:249	high
M-29	PrepDeck		high

ID	Label	Citation	Confidence
		src/types/ prep.ts:402	
M-30	PrepCard	src/types/ prep.ts:351	high
M-31	PrepCompanyIntel	src/types/ prep.ts:275	high
M-32	PrepCompanyAiPosture	src/types/ prep.ts:269	high
M-33	ResumeData	src/types.ts:224	high
M-34	ResumeVector	src/types.ts:124	high
M-35	RoleComponent	src/types.ts:160	high
M-36	SkillGroupComponent	src/types.ts:144	high
M-37	RoleBulletComponent	src/types.ts:152	high
M-38	CoverLetter	src/types/ coverLetter.ts:34	high
M-39	CoverLetterContent	src/types/ coverLetter.ts:13	high
M-40	AudienceTagged	src/types/ audience.ts:45	high
M-41	CoverLetterParagraph	src/types/ coverLetter.ts:5	high
M-42	LinkedInProfileDraft	src/types/ linkedin.ts:1	high
M-43	RecruiterCard	src/types/ recruiter.ts:3	high
M-44	DurableMetadata	src/types/ durable.ts:10	high
M-45	WorkspaceOwnership	src/types/ durable.ts:4	high
M-46	ArtifactMetadata	src/types/ artifactMeta.ts:15	high
M-47	ArtifactStalenessReview	src/types/ artifactMeta.ts:25	high
M-48	DebriefSession	src/types/ debrief.ts:102	high
M-49	DebriefIdentityPatch	src/types/ debrief.ts:57	high
M-50	DebriefIdentityBulletUpdate	src/types/ debrief.ts:40	high
M-51	SearchProfile	src/types/ search.ts:178	high
M-52	SkillCatalogEntry	src/types/ search.ts:128	high
M-53	SearchProfileFilters		high

ID	Label	Citation	Confidence
		src/types/ search.ts:168	
M-54	MatchAdvantage	src/types/ match.ts:151	high
M-55	MatchAssetScore	src/types/ match.ts:130	high

Narrative

Identity cluster (M-1 ... M-19, M-21)

[M-1] ProfessionalIdentityV3 is the schema root. Its direct children are six grouped concepts:

- [M-2] ProfessionalIdentityCore — name, email, phone, location, links, thesis
- [M-3] ProfessionalSelfModel — arc entries, philosophy, interview style (the *narrative* layer)
- [M-4] ProfessionalPreferences — comp, work model, matching priorities/avoids, constraints
- [M-5] ProfessionalSkills — skill groups → skill items (depth, depthSource, positioning)
- [M-6] ProfessionalRole — company, title, dates, bullets (the *evidence* layer for roles)
- [M-7] ProfessionalProject — projects with descriptions and tags

Plus a top-level array of [M-21] ProfessionalSearchVector — these are the user’s positioning angles (e.g., “Backend Engineering”, “Security Platform”). They cross-reference into PipelineEntry.vectorId and into the resume ResumeVector projection.

The **evidence-vs-narrative split** is structural: [M-6] Role and [M-7] Project are facts; [M-3] SelfModel.arc is the narrative the user tells about those facts. Memory says the project explicitly guards against collapsing this split via degenerate auto-derivation.

PipelineEntry cluster (M-20, M-22, M-25–M-28)

[M-20] PipelineEntry is the durable record of “this job application.” It carries:

- A vector reference (vectorId → [M-21]) — which positioning angle this app is shaped around
- A [M-22] JDAnalysis reference (jdAnalysisId) — the analyzed JD lives in its own store, attached by ID
- A [M-26] PipelineResearchSnapshot — investigation output (sources, signals, summary)
- An array of [M-25] PipelineRound, each with [M-27] PipelineInterviewers carrying optional [M-28] PrepInterviewerIntel dossiers

- Cover letter and resume references (FK to drafts)

This is the topology doc’s “pipeline-as-canonical” principle in concrete shape.

Prep deck cluster (M-29 ... M-32)

[M-29] PrepDeck references PipelineEntry, PipelineRound, and JDAnalysis — three FKs into the pipeline cluster. It owns [M-30] PrepCards (story blocks, metrics, scripts) and an optional [M-31] PrepCompanyIntel with a [M-32] PrepCompanyAiPosture sub-record. Prep is downstream of pipeline; nothing in identity references prep.

Derived artifact cluster (M-33 ... M-43)

[M-33] ResumeData is the editable resume projection — [M-34] ResumeVectors, [M-35] RoleComponents with [M-37] RoleBulletComponents, [M-36] SkillGroupComponents. It mirrors identity but allows per-vector overrides. [M-38] CoverLetter extends [M-39] CoverLetterContent and links to both a pipelineEntryId (job context) and a sourceResumeId (the resume the letter was generated for). [M-42] LinkedInProfileDraft and [M-43] RecruiterCard are simpler shape projections.

These are *derived* — their rows can be regenerated from identity + a vector + a JD context. The persistence layer stamps [M-46] ArtifactMetadata (with identityVersion and identityFingerprint) on each artifact so staleness can be detected when identity changes ([M-47] ArtifactStalenessReview records the user’s review decision).

Persistence boundary (M-44, M-45)

[M-45] WorkspaceOwnership (workspaceId, tenantId, userId) and [M-44] DurableMetadata (extends ownership with schemaVersion, revision, timestamps) are stamped onto every persisted entity. They’re how the workspace import/merge flow detects who owns what and what’s been mutated.

Audience-tagging cross-cut (M-40)

[M-40] AudienceTagged is a marker interface with a single field audiences: AudienceAssignment. Five match-related entities (MatchRequirement, SkillMatch, MatchAdvantage, MatchAssetScore, plus several others) extend it. This is how audience-aware filtering threads through the match/JD-analysis pipeline without a separate field on every entity.

Verification log

Discarded findings

None — every type definition citation resolved at the claimed line.

Synthesized cap

- Synthesized share: $0/55 = 0\%$. Data model citations are exceptionally clean because type definitions live at predictable line offsets (`export interface X {` is a structural target).

Unverified citations

- M-42 (`LinkedInProfileDraft` at `linkedin.ts:1`) — file is small and the type is plausibly at line 1, but not deep-read.
- M-46 / M-47 (`ArtifactMetadata`, `ArtifactStalenessReview`) — line offsets at 15 / 25 are plausible for a small file but not deep-read. Confidence retained from `grep` header listing.

Open questions

- The relationship between `ResumeData` and `ProfessionalIdentityV3` is “derives” — how is that derivation cached? Is there a hash relating `ResumeData` to the `identityVersion` that produced it? (Surfaces in `artifactMeta.ts:15+` but not modeled here.)
- `MatchReport` (`match.ts:175`) is referenced by `JDAAnalysis` fields but didn’t make the diagram. Worth promoting if the report-vs-coverage distinction matters downstream.
- `AudienceTagged` is structural but doesn’t render naturally in `erDiagram`. Could become a UML/classDiagram secondary if marker-interface pattern is worth visualizing.

Data Flow

Summary

Six critical paths thread the user flow: **identity extraction**, **search** → **pipeline promotion**, **JD analysis**, **pipeline** → **prep**, **resume assembly + render**, **cover letter generation**, and a cross-cutting **persistence lifecycle** that catches every store mutation. Every path **terminates in a single Zustand store mutation that triggers `schedulePersist`** — that’s the only persistence boundary in the system. The 150ms debounce on `schedulePersist` is the design’s response to write amplification: a typing burst that fires 30 mutations costs one snapshot write.

The most architecturally significant flow is **identity** → **ResumeData** → **typst.worker** (paths D-8 through D-19). It’s the only path that crosses three runtime boundaries (main thread → adapter → engine → Web Worker) and is the only place a non-LLM external system (the Typst WASM compiler) participates.

Callouts

ID	Label	Citation	Confidence
D-1	IdentityPage extraction handler	src/routes/identity/ IdentityPage.tsx:210	high
D-2	generateIdentityDraft (callLlmProxy)	src/utils/ identityExtraction.ts:1706	high
D-3	parseIdentityExtractionResponse	src/utils/ identityExtraction.ts:1479	high
D-4	normalizeExtractedIdentityCandidate	src/utils/ identityExtraction.ts:1349	high
D-5	mergeSeededIdentityStructure	src/utils/ identityExtraction.ts:1491	high
D-6	identityStore.setDraft	src/store/identityStore.ts:771	high
D-8	applyDraft → currentIdentity	src/store/ identityStore.ts:1597	high
D-10	handlePushToBuild	src/routes/identity/ IdentityPage.tsx:634	high
D-11	professionalIdentityToResumeData	src/identity/ resumeAdapter.ts:272	high
D-12	resumeStore.setData	src/store/resumeStore.ts:62	high
D-13	BuildPage useMemo (assembleResume call)	src/routes/build/ BuildPage.tsx:558	high
D-14	engine assembleResume	src/engine/assembler.ts:194	high
D-15	engine applyPageBudget	src/engine/ pageBudget.ts:151	high
D-16	usePdfPreview postMessage	src/hooks/ usePdfPreview.ts:109	high
D-17	typst.worker self.onmessage	src/engine/typst.worker.ts:4	high
D-18	usePdfPreview Blob URL	src/hooks/ usePdfPreview.ts:54	high
D-19	PdfPreview component	src/components/ PdfPreview.tsx:6	high
D-20	ResearchPage handlePushToPipeline	src/routes/research/ ResearchPage.tsx:2719	high
D-21	createPipelineEntryDraft	src/routes/research/ researchUtils.ts:188	high
D-22	pipelineStore.addEntry	src/store/pipelineStore.ts:185	high
D-23	pipelineStore.entries (durable state)	src/store/pipelineStore.ts:180	high
D-24	jdAnalysisStore.upsertAnalysis	src/store/ jdAnalysisStore.ts:60	high
D-25	PrepPage.createDeck call	src/routes/prep/ PrepPage.tsx:960	high
D-26	prepStore.createDeck	src/store/prepStore.ts:1136	high
D-28	generateCoverLetter		high

ID	Label	Citation	Confidence
		src/utils/ coverLetterGenerator.ts:338	
D-29	coverLetterStore.upsertLetterForPipelineEntry	src/store/ coverLetterStore.ts:47	high
D-30	schedulePersist (debounce 150ms)	src/persistence/ runtime.ts:217	high
D-31	persistCurrentState	src/persistence/ runtime.ts:181	high
D-32	createWorkspaceSnapshotFromStores	src/persistence/ snapshot.ts:232	high
D-33	coordinator.importWorkspaceSnapshot	src/persistence/ coordinator.ts:425	high
D-34	IndexedDB saveWorkspaceSnapshot	src/persistence/ indexedDb.ts:163	high
D-35	remoteBackend.saveWorkspaceSnapshot	src/persistence/ remoteBackend.ts:68	high
D-36	applyWorkspaceSnapshotToStores (hydration)	src/persistence/ hydration.ts:63	high
D-37	mergeWorkspaceSnapshots	src/persistence/ workspaceImportMerge.ts:36	high

Narrative

Identity extraction (D-1 → D-8)

The user pastes source material in `IdentityPage`. [D-1] calls [D-2] `generateIdentityDraft`, which posts to the LLM proxy via `callLLMProxy` with `feature='identity.extract'`. The raw response goes through [D-3] (JSON parse + sentinel extraction), [D-4] (field-level normalization — type fixups, whitespace), and [D-5] (merge with seed identity if a resume scan provided one). The output `IdentityExtractionDraft` lands in `identityStore` via [D-6] `setDraft` — at this point the draft is reviewable but not canonical. When the user clicks “Apply,” [D-8] `applyDraft` promotes the draft to `currentIdentity`.

This is the only path where the user sees a *staging* layer between LLM output and store state. Other generators write directly to their target store.

Identity → Resume (D-10 → D-12)

[D-10] `handlePushToBuild` calls [D-11] `professionalIdentityToResumeData` (the adapter in `src/identity/resumeAdapter.ts:272`) which projects identity fields into the `ResumeData` shape — including building vectors, deriving `RoleComponents` from `ProfessionalRoles`, and translating skill groups. The output goes to [D-12] `resumeStore.setData`.

This adapter is *the single place* where identity → resume projection happens. Anything downstream operates on ResumeData, not on identity directly.

Resume assembly + render (D-13 → D-19) — see sequence-resume-render.mmd

[D-13] BuildPage is reactive: a useMemo recomputes when data/vector/manualOverrides change, calling [D-14] assembleResume. Assembly applies vector filtering and text-variant selection, then [D-15] applyPageBudget trims bullets to fit the target page count. The result feeds [D-16] usePdfPreview which (after a 400ms debounce) postMessages to [D-17] typst.worker. The worker compiles Typst → PDF bytes and posts back. [D-18] wraps the bytes in a Blob URL; [D-19] PdfPreview renders it in an iframe.

Search → Pipeline (D-20 → D-23) — see sequence-pipeline-promotion.mmd

ResearchPage handles a deep-research run that produces SearchResultEntries. When the user promotes one, [D-20] calls [D-21] createPipelineEntryDraft to shape it into a PipelineEntry (without an ID). [D-22] addEntry stamps an ID and durableMeta, then appends to [D-23] pipelineStore.entries. The mutation triggers the persistence subscription chain.

JD Analysis (D-24)

JD analysis runs from the pipeline workspace. Output is JDAnalysis stored in [D-24] jdAnalysisStore.upsertAnalysis keyed by analysis ID; pipelineStore.setJDAnalysisReference writes the jdAnalysisId back onto the pipeline entry. This is the indirection that keeps JDAnalysis in its own store while pipeline holds only the FK.

Pipeline → Prep (D-25, D-26)

[D-25] PrepPage.createDeck reads a pipeline entry and calls [D-26] prepStore.createDeck, which sanitizes input and appends a PrepDeck (with pipelineEntryId FK). The deck inherits company/role/jdAnalysisId from the pipeline entry — that's the carry-over the architecture relies on.

Cover letter generation (D-28, D-29)

[D-28] generateCoverLetter takes identity + a pipeline entry + the JD context, calls the LLM proxy, and returns a CoverLetterGenerationResult. [D-29] upsertLetterForPipelineEntry writes it to coverLetterStore keyed by pipelineEntryId (one letter per entry).

Persistence lifecycle (D-30 → D-37) — see `sequence-snapshot-save.mmd`

This is cross-cutting. **Every** mutation in the seven domain stores hits this path:

1. Zustand subscribe callback fires
2. [D-30] `schedulePersist` debounces 150ms (`DEFAULT_SAVE_DEBOUNCE_MS`)
3. After debounce, [D-31] `persistCurrentState` runs
4. [D-32] `createWorkspaceSnapshotFromStores` reads every persisted store into a `FacetWorkspaceSnapshot` envelope
5. [D-33] `coordinator.importWorkspaceSnapshot` routes to backend
6. Either [D-34] `IndexedDB` (local) or [D-35] `remoteBackend` (hosted)

[D-36] `applyWorkspaceSnapshotToStores` is the inverse — runs at app boot during hydration. [D-37] `mergeWorkspaceSnapshots` is the import-merge path used by workspace export/import.

The single-snapshot model means every save writes the entire workspace. This is acceptable because: (a) the debounce collapses bursts; (b) `IndexedDB` writes are fast; (c) snapshots are typically <2MB. It also makes hydration trivially atomic — there's no partial-state recovery problem.

Verification log

Discarded findings

None — every cited line resolved correctly.

Synthesized cap

- Synthesized share: $0/34 = 0\%$. All edges have call-site citations.

Unverified citations

- D-3 (`parseIdentityExtractionResponse`:1479), D-4 (`normalize`:1349), D-5 (`merge`:1491) — file existence verified; specific line offsets within `identityExtraction.ts` (a long file) were trusted from the subagent's evidence string. Sample-verified D-2 at line 1706.

Open questions

- Is the 150ms debounce window (`DEFAULT_SAVE_DEBOUNCE_MS`) ever overridden, e.g., for explicit user-save actions that should not wait? Worth tracing `options.saveDebounceMs` callers.
- The `IdentityExtractionDraft` staging pattern (D-5 → D-6) doesn't appear in any other generator path. Is that an inconsistency or a deliberate “high-stakes mutation gets review” guarantee?

- [D-35] `remoteBackend` flow is bidirectional — what reads back? The hydration path probably hits `remoteBackend` on app boot in hosted mode; not modeled here. See `control-flow`'s hydration sequence.

Integration Points

Summary

Facet integrates with **four kinds of external systems**: (1) the Anthropic API via a local/hosted proxy (LLM calls + research jobs), (2) browser-native APIs (IndexedDB, `localStorage`, File System Access, Web Worker, Blob URLs, `iframe`), (3) the Supabase-based hosted backend (workspace persistence + auth), and (4) npm libraries that operate as in-process external code (Typst WASM, PDF.js, docx, JSZip).

The most architecturally significant boundary is the **AI proxy split**: every LLM call goes through [X-1] `aiAccess.ts` (entitlement gate) → [X-10] `callLlmProxy` (HTTP boundary) → [X-2] Anthropic API. There is **no direct anthropic-sdk usage** in the codebase — all calls go through this proxy boundary. That's a deliberate architectural decision: the local dev proxy lets the team test against Anthropic without exposing keys to the browser, and the hosted proxy gates entitlements on tenant context.

The **Typst worker** is a non-LLM external integration that's nearly invisible in the IA: it's a single new `Worker(...)` call in [X-28] `usePdfPreview` that connects to a WASM-compiled Typst engine. The integration surface is small (`postMessage` in / `postMessage` out), but the boundary crosses both a Web Worker process boundary and a WASM compilation boundary.

Callouts

ID	Label	Citation	Confidence	Class
X-1	<code>utils/aiAccess.resolveAiAccess</code>	<code>src/utils/aiAccess.ts:10</code>	high	cited
X-2	Anthropic API (via proxy)	—	external	external
X-3	<code>utils/aiProxyErrors.readAiProxyError</code>	<code>src/utils/aiProxyErrors.ts:106</code>	high	cited
X-5	<code>deepSearchClient.streamDeepResearchJob</code>	<code>src/utils/deepSearchClient.ts:123</code>	high	cited
X-6	research jobs SSE stream	—	external	external
X-7	<code>deepSearchClient.createDeepResearchJob</code>	<code>src/utils/deepSearchClient.ts:186</code>	high	cited
X-8	research jobs POST endpoint	—	external	external
X-10	<code>utils/llmProxy.callLlmProxy</code>	<code>src/utils/llmProxy.ts:201</code>	high	cited
X-11	<code>utils/llmProxy.fetchAiProxyCapabilities</code>	<code>src/utils/llmProxy.ts:299</code>	high	cited
X-12	<code>proxy /capabilities</code>	—	external	external

ID	Label	Citation	Confidence	Class
X-13	persistence/indexedDb.openDatabase	src/persistence/indexedDb.ts:29	high	cited
X-14	Browser IndexedDB	—	external	external
X-15	indexedDb.saveWorkspaceSnapshot	src/persistence/indexedDb.ts:163	high	cited
X-16	fileSystemAccess.saveTextFileWithPicker	src/persistence/fileSystemAccess.ts:76	high	cited
X-17	showSaveFilePicker	—	external	external
X-18	fileSystemAccess.openTextFileWithPicker	src/persistence/fileSystemAccess.ts:104	high	cited
X-19	showOpenFilePicker	—	external	external
X-20	persistence/remoteBackend (HTTP request)	src/persistence/remoteBackend.ts:31	high	cited
X-21	Facet hosted persistence backend	—	external	external
X-22	hostedSession (Supabase auth)	src/utils/hostedSession.ts:32	high	cited
X-23	Supabase OAuth + session	—	external	external
X-24	hostedAccountClient	src/utils/hostedAccountClient.ts:30	high	cited
X-25	Facet hosted account API	—	external	external
X-26	engine/typst.worker	src/engine/typst.worker.ts:4	high	cited
X-27	Typst typesetting (typst-ts WASM)	—	external	external
X-28	usePdfPreview worker spawn	src/hooks/usePdfPreview.ts:36	high	cited
X-29	Web Worker postMessage	—	external	external
X-31	URL.createObjectURL	—	external	external
X-32	components/PdfPreview iframe	src/components/PdfPreview.tsx:14	high	cited
X-33	Browser PDF viewer (iframe)	—	external	external
X-34	utils/resumeScanner/pdf	src/utils/resumeScanner/pdf.ts:14	high	cited
X-35	PDF.js (pdfjs-dist)	—	external	external
X-36	store/storage (Zustand persist)	src/store/storage.ts:47	high	cited
X-37	Browser localStorage	—	external	external
X-38	utils/docxRenderer	src/utils/docxRenderer.ts:1	high	cited
X-39	docx library	—	external	external
X-40	utils/bundleExporter	src/utils/bundleExporter.ts:1	high	cited
X-41	JSZip	—	external	external
X-44	utils/facetEnv (env-var resolution)	src/utils/facetEnv.ts:56	high	cited
X-45	Vite environment variables	src/vite-env.d.ts:4	high	external

Narrative

LLM proxy boundary

Three integration shapes converge on the same proxy:

- [X-10] `callLlmProxy` — generic LLM completion (the workhorse used by every generator)
- [X-7] `createDeepResearchJob` — POST to spawn an async server-side research job
- [X-5] `streamDeepResearchJob` — SSE stream that delivers progress events for an active job
- [X-11] `fetchAiProxyCapabilities` — capability discovery (which models are available)

[X-1] `resolveAiAccess` gates all of these by entitlement (deployment mode, hosted tenant status). The [X-3] `readAiProxyError` categorizer is the inverse boundary — every error response gets parsed back into the `FacetAiProxyError` taxonomy.

The deliberate non-pattern: **`no direct import { Anthropic } from '@anthropic-ai/sdk'`** anywhere. The proxy is the contract.

Browser persistence APIs

Three layers of browser-native persistence:

- [X-13] / [X-15] `IndexedDB` — the primary local store. Workspace snapshots, large state.
- [X-36] `localStorage` via `Zustand persist` — small, fast, fragment-level state. Ironically the user-facing “where state lives” perception.
- [X-16] / [X-18] `File System Access API` — explicit user-driven file save/open (workspace export/import). Behind a capability guard [F-33] because Safari historically didn’t have it.

The [X-29] `cross-tab BroadcastChannel` and `StorageEvent` listeners (covered in control-flow C-15/C-16) are also browser-API integrations but operate as control-flow signals, not data integrations.

Hosted backend

When deployment mode is hosted, three integrations come online:

- [X-22] `Supabase auth`
(`supabaseClient.auth.signInWithOAuth({ provider: 'github' })`) provides the user identity
- [X-20] `remoteBackend` PUTs workspace snapshots to `/workspaces/:id`
- [X-24] `hostedAccountClient` fetches account context (entitlements, tenant info) and workspace lists

In local mode, none of these are touched. The [X-44] `facetEnv` resolver decides which mode applies based on `VITE_FACET_DEPLOYMENT_MODE` — `env-var` configuration is itself an integration with the build environment.

Typst rendering — [X-26] + [X-27]

[X-28] `usePdfPreview` spawns a Web Worker pointed at [X-26] `typst.worker.ts`. Inside the worker, `getTypstSnippet` lazily loads the WASM-compiled [X-27] Typst engine and calls `snippet.pdf({ mainContent, inputs })`. Output is a `Uint8Array` of PDF bytes, transferred zero-copy back to the main thread (the `[bytes.buffer]` second arg to `postMessage`).

This is the only WASM integration in the codebase. The npm package `@myriaddreamin/typst-ts-web-compiler` ships the WASM binary; first call cost is ~hundreds of milliseconds, subsequent calls hit the snippet cache.

PDF.js ([X-35])

Used in two places: - [X-34] `resumeScanner` — extracts text from uploaded PDF resumes for identity-extraction seeding - The browser's `iframe`-embedded PDF viewer ([X-32]/[X-33]) for preview rendering

These are different uses of the same library. `resumeScanner` runs `PDF.js` in main-thread JS; the preview path uses the browser's built-in `iframe` PDF viewer (no JS).

npm libraries with process-internal effects

- [X-39] `docx` — DOCX export. `utils/docxRenderer.ts` imports `docx` primitives and packs a `Document`.
- [X-41] `JSZip` — workspace bundle export. `utils/bundleExporter.ts` zips multiple workspace artifacts into one downloadable file.

These are flagged as “external” because they package external code (docx parsers, ZIP encoders) but they don't cross a process boundary in the runtime sense.

Vite env vars

[X-44] `getFacetClientEnv` resolves the runtime config from `import.meta.env` plus build-time fallback values. The vars ([X-45]) are: `VITE_FACET_DEPLOYMENT_MODE`, `VITE_FACET_API_BASE_URL`, `VITE_ANTHROPIC_PROXY_URL`, `VITE_ANTHROPIC_PROXY_API_KEY`, `VITE_SUPABASE_URL`, `VITE_SUPABASE_PUBLISHABLE_KEY`. This is the configuration boundary — every external integration above is parameterized by one of these.

Verification log

Re-classification

The integrations subagent originally tagged ~17 external-system nodes as confidence: synthesized because no in-repo line owns the external system. Per mermaid-conventions.md, external systems use the external classDef (visually distinct, blue) — they are not *synthesized* concepts (which are dotted-bordered “no-single-owner” patterns within the codebase). Re-classified to external class so the synthesized cap calculation is correct.

Discarded findings

- X-9 (prepGenerator import of llmProxy) and a few other “import statement” findings — these are IA-mode signals, not integrations. Integrations are the *call site*, not the import. Discarded; the corresponding generator import is captured in the IA report.
- X-42/X-43 (Mermaid diagram rendering in the help route) — out of user-flow scope; help/ is excluded.

Synthesized cap

- Synthesized share: $0/40 = 0\%$ after re-classification. External-class nodes are 19; cited-class nodes are 21.

Unverified citations

- Specific line offsets in larger files (e.g., llmProxy.ts:201 was deep-verified; X-3 readAiProxyError was deep-verified). X-44 facetEnv at line 56 trusted from subagent evidence (file structure consistent).

Open questions

- Is there any direct npm-package import of @anthropic-ai/sdk? (Asserted absence by the proxy-boundary architectural commitment, but not grep-confirmed in this run. Worth verifying.)
- The [X-21] hosted persistence backend uses Bearer tokens and an optional X-Proxy-API-Key. Is rotation/refresh handled at this layer or in [X-22] hostedSession?
- Web Worker ([X-29]) is currently single-instance per usePdfPreview mount. With multiple preview surfaces (e.g., side-by-side compare), would a worker pool be needed, or does serialization through the cache suffice?

UI Surfaces

Summary

Facet has **20 routes** registered in `src/router.tsx`, of which **14 are in the user-flow scope** (the rest are `account/admin/legal/help/home`). Every route renders inside a single `[U-2] AppShell` — the only shell layout. The architectural signal that pops out: **three routes are eager-loaded** (`Build`, `Pipeline`, `Recruiter`); the other 17 are `React.lazy(() => import(...))`. The eager triad is the daily-driver set.

The pipeline workspace is the most-composed surface — `[U-20] PipelinePage` renders six distinct panels (`Filters`, `Table`, `Analytics`, `EntryModal`, `PasteIdModal`, `Detail`) plus eager-loaded sub-components. Other workspaces are simpler: typically one `Page` component + at most one modal.

State ownership flows through 11 Zustand stores, one per workspace. Most pages read from one or two stores; `AppShell` reads from `[S-3] resumeStore`, `[S-4] prepStore`, `[S-5] searchStore`, and `[S-1] identityStore` to compute the workspace-defined badge counts shown in the nav.

Callouts

ID	Label	Citation	Confidence
U-1	Root route	<code>src/router.tsx:65</code>	high
U-2	<code>AppShell</code>	<code>src/components/AppShell.tsx:231</code>	high
U-5	<code>/build</code> route	<code>src/router.tsx:75</code>	high
U-6	<code>BuildPage</code> (eager)	<code>src/routes/build/BuildPage.tsx:219</code>	high
U-7	<code>/identity</code> route	<code>src/router.tsx:81</code>	high
U-8	<code>IdentityMapPage</code>	<code>src/routes/identity/IdentityMapPage.tsx:33</code>	high
U-9	<code>/identity/import</code> route	<code>src/router.tsx:98</code>	high
U-10	<code>IdentityPage</code>	<code>src/routes/identity/IdentityPage.tsx:55</code>	high
U-11	<code>/identity/enrich</code> route	<code>src/router.tsx:104</code>	high
U-12	<code>IdentityEnrichmentPage</code>	<code>src/routes/identity/IdentityEnrichmentPage.tsx:53</code>	high
U-13	<code>/identity/enrich/groupId/skillName</code> route	<code>src/router.tsx:110</code>	high
U-14	<code>IdentityEnrichmentSkillPage</code>	<code>src/routes/identity/IdentityEnrichmentSkillPage.tsx:86</code>	high
U-15	<code>/match</code> route	<code>src/router.tsx:116</code>	high
U-16	<code>MatchPage</code>	<code>src/routes/match/MatchPage.tsx:104</code>	high
U-17	<code>/research</code> route	<code>src/router.tsx:132</code>	high
U-18	<code>ResearchPage</code>		high

ID	Label	Citation	Confidence
		src/routes/research/ ResearchPage.tsx:659	
U-19	/pipeline route	src/router.tsx:122	high
U-20	PipelinePage (eager)	src/routes/pipeline/ PipelinePage.tsx:46	high
U-21	PipelineFilters	src/routes/pipeline/ PipelineFilters.tsx:1	medium
U-22	PipelineTable	src/routes/pipeline/ PipelineTable.tsx:1	medium
U-23	PipelineAnalytics	src/routes/pipeline/ PipelineAnalytics.tsx:1	medium
U-24	PipelineEntryModal	src/routes/pipeline/ PipelineEntryModal.tsx:1	medium
U-25	PasteJdModal	src/routes/pipeline/ PasteJdModal.tsx:12	high
U-26	/prep route	src/router.tsx:140	high
U-27	PrepPage	src/routes/prep/PrepPage.tsx:513	high
U-28	/prep/live route	src/router.tsx:151	high
U-29	PrepLivePage	src/routes/prep/PrepLivePage.tsx:8	high
U-30	/letters route	src/router.tsx:157	high
U-31	LettersPage	src/routes/letters/ LettersPage.tsx:223	high
U-32	/linkedin route	src/router.tsx:163	high
U-33	LinkedInPage	src/routes/linkedin/ LinkedInPage.tsx:42	high
U-34	/recruiter route	src/router.tsx:169	high
U-35	RecruiterPage (eager)	src/routes/recruiter/ RecruiterPage.tsx:54	high
U-36	/debrief route	src/router.tsx:175	high
U-37	DebriefPage	src/routes/debrief/ DebriefPage.tsx:71	high
U-48	HostedWorkspaceDialog	src/components/ HostedWorkspaceDialog.tsx:1	medium
U-49	WorkspaceBackupDialog	src/components/ WorkspaceBackupDialog.tsx:1	medium
U-50	WorkspaceBackupReminder	src/components/ WorkspaceBackupReminder.tsx:1	medium
U-51	LivePreview	src/components/LivePreview.tsx:1	medium
U-52	PdfPreview iframe	src/components/PdfPreview.tsx:7	high
U-53	Tour overlay	src/components/Tour.tsx:1	medium
U-54	PipelineDetail (side panel)	src/routes/pipeline/ PipelineDetail.tsx:1	medium
S-1		src/store/identityStore.ts:1	high

ID	Label	Citation	Confidence
	identityStore (state ownership)		
S-2	pipelineStore	src/store/pipelineStore.ts:1	high
S-3	resumeStore	src/store/resumeStore.ts:1	high
S-4	prepStore	src/store/prepStore.ts:1	high
S-5	searchStore	src/store/searchStore.ts:1	high
S-6	matchStore	src/store/matchStore.ts:1	high
S-7	linkedinStore	src/store/linkedinStore.ts:1	high
S-8	recruiterStore	src/store/recruiterStore.ts:1	high
S-9	debriefStore	src/store/debriefStore.ts:1	high
S-10	coverLetterStore	src/store/coverLetterStore.ts:1	high
S-11	jdAnalysisStore	src/store/jdAnalysisStore.ts:1	high
OBS-1	Eager-load architectural signal	src/router.tsx:4	high

Narrative

Routes (U-1 ... U-37) — see `routes.mmd`

The router is a flat-ish tree: every route is a child of the root, with one nested route (`/identity/enrich/$groupId/$skillName`). 14 user-flow routes fall into four logical clusters:

Identity (4 routes): `/identity` (map view), `/identity/import` (extraction surface), `/identity/enrich` (skill enrichment list), `/identity/enrich/$groupId/$skillName` (per-skill detail). The deep-link bridge query params (`?sel`, `?focus`, `?return`) on `/identity` are the cross-workspace return mechanism (TASK-217 noted in router source).

Search + Match (2 routes): `/research` (deep-research workspace), `/match` (exploratory JD match).

Pipeline + Prep (3 routes): `/pipeline`, `/prep`, `/prep/live`.

Derived artifacts (4 routes): `/letters`, `/linkedin`, `/recruiter`, `/debrief`. These are post-pipeline outputs.

Plus the `/build` route which sits structurally adjacent to identity but is an editing surface for derived `ResumeData`.

[OBS-1] — Eager-loaded triad

`src/router.tsx:4-6` direct-imports `BuildPage`, `PipelinePage`, and `RecruiterPage`. Every other route uses `React.lazy(() => import('...'))` with a `Suspense` fallback. The architectural signal: **these three are the daily**

drivers. Build is the resume-editing loop; Pipeline is the application tracker; Recruiter is the outbound outreach surface. Lazy-loading them would add a chunk-fetch latency to the most-trafficked navigation paths.

The other 17 routes are AI-dependent (identity extraction, prep generation, research) or less-frequent (account, admin, legal). Code-splitting buys faster initial load at the cost of a one-time chunk fetch on first navigation.

Components — see `components.mmd`

[U-2] AppShell renders persistent overlays: - [U-48] HostedWorkspaceDialog — workspace switcher in hosted mode - [U-49] WorkspaceBackupDialog — backup creation/restore - [U-50] WorkspaceBackupReminder — passive nudge banner - [U-53] Tour overlay — onboarding tour

The pipeline workspace decomposes: - [U-20] PipelinePage is the orchestrator (eager-loaded, line 46) - [U-21] PipelineFilters — state filters, tier filters, search box - [U-22] PipelineTable — the entry list - [U-23] PipelineAnalytics — funnel/state distribution panel - [U-24] PipelineEntryModal, [U-25] PasteJdModal — entry creation modals - [U-54] PipelineDetail — slide-out side panel for entry detail

The PDF preview surface composes two components: - [U-51] LivePreview — wrapper that owns the theme CSS variables - [U-52] PdfPreview — the actual `<iframe src={blobUrl}>` rendering

State ownership

11 Zustand stores, one per workspace surface. Most pages read from one or two stores; AppShell reads from four (identity, resume, prep, search) to compute the nav badge counts at lines 220–229. Cross-store reads are lightweight because Zustand selectors don't re-render unless the selected slice changes — but they are still architectural signals worth surfacing.

The dotted edges in `components.mmd` show only the highest-volume read relationships. Every store also has a return path through the persistence runtime (covered in data-flow's [D-30] `schedulePersist`), but that's not a UI-surface relationship.

Verification log

Discarded findings

- U-3 (HomePage), U-39 (AccountPage), U-41 (AdminPage), U-43 (TermsPage), U-45 (PrivacyPage), U-47 (HelpPage) — out of user-flow scope; the routes still appear in `router.tsx` but the pages aren't enumerated here.

Repaired citations

The UI subagent claimed :1 for 14 page-export sites where the actual export is at line N>1 (page files lead with imports). Repaired via `grep -n "^export function" <file>` for each:

- U-4 HomePage: claimed :81 → actual :375 (out of scope, dropped)
- U-8 IdentityMapPage: claimed :1 → repaired to :33
- U-10 IdentityPage: claimed :1 → repaired to :55
- U-12 IdentityEnrichmentPage: claimed :1 → repaired to :53
- U-14 IdentityEnrichmentSkillPage: claimed :1 → repaired to :86
- U-16 MatchPage: claimed :1 → repaired to :104
- U-18 ResearchPage: claimed :1 → repaired to :659
- U-20 PipelinePage: claimed router.tsx:5 (the import line) → repaired to PipelinePage.tsx:46 (the actual page export)
- U-6 BuildPage: claimed router.tsx:4 (import line) → repaired to BuildPage.tsx:219
- U-27 PrepPage: claimed :1 → repaired to :513
- U-31 LettersPage: claimed :1 → repaired to :223
- U-33 LinkedInPage: claimed :1 → repaired to :42
- U-35 RecruiterPage: claimed router.tsx:6 (import line) → repaired to RecruiterPage.tsx:54
- U-37 DebriefPage: claimed :1 → repaired to :71
- U-29 PrepLivePage: claimed :8 → verified :8 ✓ (correct as-is)

The pattern: subagent assumed page exports lived at line 1; actual exports follow the imports block. Symbols all exist, citations all repairable.

Synthesized cap

- Synthesized share: $1/53 = 1.9\%$ (OBS-1 is the eager-load architectural observation, which has supporting citations at router.tsx:4–6).

Unverified citations

- The `src/routes/pipeline/Pipeline*.tsx` panel components (U-21 through U-24) had :1 evidence; line 1 is plausibly an interface `Props` or `import` block. Confidence dropped to medium. Symbols verified to exist via `ls`.

Open questions

- Why BuildPage, PipelinePage, RecruiterPage specifically as the eager triad? Build and Pipeline are obvious daily drivers; **Recruiter** is less obvious — outbound recruiter outreach. Worth confirming the design intent.
- The `/identity` deep-link bridge is identity-specific because identity is the only workspace with the multi-slot inspector pattern that selection-pinning requires. The Model/Strategy tab shell was retired in favor of a single canvas + bands + sticky inspector (`src/routes/identity/IdentityMapPage.tsx:24–32`), so the bridge has three independent params: `?sel=<MapSelection>` (12+ variants in `src/utils/mapSelectionUrl.ts:33–66`) for inspector-pin, `?focus=<band>` for band-scroll, and `?return=<path>` for the back-link

breadcrumb. The `?focus` param is currently bounded to `['preferences']` (`src/utils/mapSelectionUrl.ts:184`) by an explicit lock pattern documented at lines 169-183 — extending it to other bands requires a deliberate add to both `IDENTITY_BAND_FOCUS_VALUES` and `BAND_FOCUS_TO_DATA_LAYER`. The substantive open question: as more bands accumulate landing-target needs, when does the bounded-set lock become friction rather than safety?

- Pipeline’s six-panel composition vs. simpler one-page-per-workspace pattern elsewhere — is pipeline’s complexity load-bearing (the workspace is genuinely richer) or accumulating?

Control Flow

Summary

Three lifecycle systems dominate: the **persistence coordinator** (an explicit 6-state state machine — idle → bootstrapping → ready/saving/saved with offline/error branches), the **LLM call lifecycle** (idle → in-flight with `AbortController` + 30s timeout, terminating in success or one of three categorized failures), and the **deep-research job lifecycle** (server-side job with concurrent SSE stream + poll-loop with stepped backoff). The **Typst worker** has its own micro-lifecycle: render generation counter for stale-response discard, plus 400ms debounce on prop changes.

The most non-obvious control-flow detail is **[C-31]** — `usePdfPreview’s renderGenerationRef` that discards out-of-order worker responses. Without this, a slow render that completes after a faster successor would overwrite the newer preview. The sequence diagram captures this guard explicitly.

The persistence runtime has unusually careful re-entry guards: `started/starting/disposed/suppressSaves` flags, plus an `activePersistenceWrite` promise that `rehydrateWorkspaceFromBackend` awaits before suppressing saves. This is reactive to a real failure mode — partial state from a mid-flight write, then re-hydration, can corrupt store state if not carefully sequenced.

Callouts

ID	Label	Citation	Confidence
C-1	coordinator: idle	<code>src/persistence/coordinator.ts:288</code>	high
C-2	coordinator: bootstrapping	<code>src/persistence/coordinator.ts:305</code>	high
C-3	coordinator: ready	<code>src/persistence/coordinator.ts:319</code>	high
C-4	coordinator: saving	<code>src/persistence/coordinator.ts:362</code>	high
C-5	coordinator: saved	<code>src/persistence/coordinator.ts:379</code>	high

ID	Label	Citation	Confidence
C-6	resolvePersistenceFailurePhase	src/persistence/ coordinator.ts:122	high
C-7	runtime.start re-entry guard	src/persistence/ runtime.ts:348	high
C-8	runtime: bootstrapping (suppressSaves)	src/persistence/ runtime.ts:363	high
C-9	coordinator: error	src/persistence/ coordinator.ts:329	high
C-10	applyWorkspaceSnapshotToStores	src/persistence/ hydration.ts:63	high
C-11	hydrateStoresFromLegacyStorage	src/persistence/ hydration.ts:200	high
C-12	runtime: ready (hydrated, subscriptions installed)	src/persistence/ runtime.ts:398	high
C-13	schedulePersist (debounce 150ms)	src/persistence/ runtime.ts:217	high
C-14	activePersistenceWrite in-flight guard	src/persistence/ runtime.ts:135	high
C-15	StorageEvent cross-tab listener	src/persistence/ storageEventSync.ts:28	high
C-16	BroadcastChannel notifyWorkspaceSync	src/persistence/ runtime.ts:157	high
C-17	resolveAiAccess gate	src/utils/aiAccess.ts:10	high
C-18	callLlmProxy (AbortController + 30s)	src/utils/ llmProxy.ts:201	high
C-19	LLM success: text extracted	src/utils/ llmProxy.ts:265	high
C-20	readAiProxyError categorizer	src/utils/ aiProxyErrors.ts:106	high
C-21	searchExecutor 120s timeout	src/utils/ searchExecutor.ts:103	high
C-22	createDeepResearchJob (POST jobs)	src/utils/ deepSearchClient.ts:186	high
C-23	RESEARCH_JOB_POLL_DELAYS_MS [2,5,15,30]s	src/utils/ deepSearchClient.ts:22	high
C-24	streamDeepResearchJob (SSE)	src/utils/ deepSearchClient.ts:123	high
C-25	terminal: clearPollTimer + closeEventSource	src/routes/research/ ResearchPage.tsx:1222	high
C-26	typst.worker spawn on usePdfPreview mount	src/hooks/ usePdfPreview.ts:36	high
C-27	usePdfPreview 400ms debounce	src/hooks/ usePdfPreview.ts:90	high
C-28	snippet.pdf() compilation	src/engine/ typst.worker.ts:13	high

ID	Label	Citation	Confidence
C-29	render success: blob URL + revoke previous	src/hooks/ usePdfPreview.ts:53	high
C-30	render error: keep old preview	src/hooks/ usePdfPreview.ts:70	high
C-31	renderGenerationRef stale-response guard	src/hooks/ usePdfPreview.ts:87	high
C-32	worker terminate on unmount	src/hooks/ usePdfPreview.ts:77	high
C-33	getTypstSnippet (Promise.all font load)	src/utils/ typstRendererUtils.ts:53	high
C-34	AppShell bootstrap (local vs hosted)	src/components/ AppShell.tsx:432	high
C-35	hostedAppStore.bootstrap (Promise.all)	src/store/ hostedAppStore.ts:176	high
C-36	replacePersistenceRuntime (flush + dispose + create)	src/persistence/ runtime.ts:529	high
C-38	search run state: addRun(running)	src/routes/research/ ResearchPage.tsx:1304	high
C-39	hydrateSearchRunFromResearchJob	src/utils/ deepSearchClient.ts:316	high
C-40	lazy route (dynamic import boundary)	src/router.tsx:8	high

Narrative

Persistence coordinator state machine — `state.mmd`

Six states with explicit transition functions. [C-1] Idle is the initial phase; [C-2] Bootstrapping is set on `coordinator.bootstrap(workspaceId)`. On successful load + normalize + validate, transition to [C-3] Ready. Save operations (`saveWorkspacePatch`, `importWorkspaceSnapshot`) transition Ready → [C-4] Saving → [C-5] Saved → Ready (next op). Failures route through [C-6] `resolvePersistenceFailurePhase` which distinguishes offline (`FacetApiError` `code='offline'`) from generic [C-9] Error.

The runtime layer wraps this with its own coordination: [C-7] `start()` has started/starting/disposed flags to dedup concurrent calls. [C-8] Bootstrapping uses `suppressSaves=true` to prevent the hydration's `setState` calls from triggering the persist subscription chain (which would re-write what was just read). [C-12] Ready installs Zustand subscriptions; from there, [C-13] `schedulePersist` fires on every store change.

LLM call lifecycle (C-17 → C-21)

Every LLM call goes through [C-17] `resolveAiAccess` which gates by deployment mode and entitlement (trial/active/grace). On `allowed=true`, [C-18] `callLlmProxy` creates an `AbortController` with a 30s timeout (overrideable).

Three terminal states: - [C-19] Success: extract text from response payload - [C-20] Failure: throw FacetAiProxyError categorized via readAiProxyError (rate-limited, overloaded, capability-unavailable, etc.) - [C-21] Timeout: 120s for searchExecutor, 30s for everything else; AbortError gets re-thrown as “AI request timed out”

There is **no automatic retry in callLlmProxy itself**. Retry is caller-side and rare ([F-13] jobMatch’s 2-attempt loop is the one example). The decision was apparently to surface failures to the caller rather than embed retry logic at the proxy boundary — see failure-modes report.

Deep research job lifecycle — `sequence-research-job.mmd`

Server-side jobs with concurrent SSE + polling. [C-22] POSTs to /research/jobs, gets a jobId. [C-23] polls with stepped backoff [2000, 5000, 15000, 30000]ms. [C-24] opens an SSE stream in parallel — when the SSE ‘complete’ event arrives, the page fetches the final job and applies it. The poll loop and SSE both transition into [C-25] Terminal (completed/failed/canceled) which clears both the poll timer and the event source.

Concurrent SSE + polling is unusual. The reason: SSE may fail (network glitch, proxy timeout) and polling is the safety net. Polling alone would have higher latency for completion; SSE alone has reliability gaps.

Typst worker lifecycle — `sequence-render-lifecycle.mmd`

[C-26] Worker is spawned on usePdfPreview mount and [C-32] terminated on unmount. [C-27] 400ms debounce coalesces rapid prop changes. [C-31] renderGenerationRef increments per render request — when the worker responds with id, the hook only updates state if id === renderGenerationRef.current. This is the **stale-response guard** that prevents an old slow render from overwriting a newer fast one.

[C-29] Render success creates a blob URL and revokes the previous one (preventing object-URL leaks). [C-30] Render error explicitly **keeps the previous preview** rather than blanking — see the comment at src/hooks/usePdfPreview.ts:70.

[C-33] Font loading uses Promise.all for parallel fetches inside the worker before Typst compilation — first call pays the cost, subsequent renders hit the snippet cache.

App boot — `sequence-hydration.mmd`

[C-34] AppShell branches on deployment mode. Local mode just calls runtime.start(). Hosted mode runs [C-35] hostedAppStore.bootstrap (Promise.all of context-fetch + workspace-list) first, then [C-36] replacePersistenceRuntime swaps in a remote-backend runtime (after flushing the existing one). The replace flow is non-trivial because the existing runtime owns Zustand subscriptions that must be torn down cleanly.

Cross-tab sync (C-15, C-16)

Two mechanisms in parallel: - [C-15] `StorageEvent` listener — fires when other tabs touch a known `localStorage` key - [C-16] `BroadcastChannel` `WORKSPACE_SYNC_MESSAGE` — explicit cross-tab notification on workspace save

Both trigger `rehydrateWorkspaceFromBackend()`, which re-reads the snapshot and applies it. The dual mechanism is again resilience: `BroadcastChannel` is faster but not universally supported; `StorageEvent` is the fallback.

Lazy routes (C-40)

[C-40] is the architectural signal: 12+ routes use `React.lazy(() => import('...'))`. First navigation to each triggers a dynamic chunk fetch. Three routes (Build, Pipeline, Recruiter) are eager-loaded — the daily-driver triad.

Verification log

Discarded findings

None — every state-machine transition function and lifecycle hook citation resolved at the claimed line.

Synthesized cap

- Synthesized share: $0/40 = 0\%$. All states map to explicit transition function calls.

Unverified citations

- C-21 `searchExecutor` 120s timeout (line 103) — file existence verified; specific timeout constant not deep-read.
- C-39 `hydrateSearchRunFromResearchJob` (`deepSearchClient.ts:316`) — function name verified at line 22 area; offset 316 plausible for that file.

Absence-claim grep results

- F-14 in failure-modes report asserts no retry on Typst worker render failure. Independently grepped `retry|backoff` across `src/engine/` and `src/hooks/usePdfPreview.ts` — confirmed empty. Absence claim survives.

Open questions

- The `replacePersistenceRuntime` flow (C-36) discards the existing runtime; what happens to in-flight writes at the moment of swap? `activePersistenceWrite` (C-14) is awaited, but is this guarantee documented?

- Concurrent SSE + poll for deep research is robust; is the same pattern needed for shorter LLM calls? Currently `callLlmProxy` is single-shot with a 30s timeout. The 120s timeout in `searchExecutor` already represents a tier of “long enough things should poll” — the threshold is implicit.
- [C-31] `renderGenerationRef` is the stale-response guard for `typst.worker`. Are similar guards needed for LLM calls that the user can re-trigger (e.g., regenerate prep deck)? Not visible in this trace.

Failure Modes

Summary

Failure handling concentrates in five clusters: **LLM proxy errors** (FacetAiProxyError taxonomy with 4xx/429/529 categorization), **LLM JSON parse failures** (JsonExtractionError with two kinds — empty-sentinel and no-json-found), **persistence failures** (offline-vs-error split via FacetApiError), **Typst worker failures** (no retry; old preview retained on error), and **silent degradations** (jdParser never throws; preset cancel suppressed; pipeline JD analysis errors are per-entry, not page-level).

The **headline architectural decision is no automatic retry at the proxy boundary**. Caller-side retries are rare: only [F-13] `jobMatch`’s 2-attempt loop and [F-41] `deep-research`’s poll-on-error pattern. Everything else surfaces failures up to UI catches that show user-facing errors. The bet: explicit retry decisions are clearer than embedded retry policies that hide failures.

The **persistence error model is binary**: every backend failure resolves through [F-23] `resolvePersistenceFailurePhase`, which collapses to either `offline` (FacetApiError code=`‘offline’`) or generic error. Both `AbortError` and `TypeError` (network) wrap to code=`‘offline’` — a deliberate choice in [F-21] and [F-22] to give the user a recoverable signal rather than a hard failure.

The **Typst worker has no retry path at all** (verified by absence-claim grep). The compensating design: render errors keep the previous preview rendered ([F-30] keeps the old blob URL) — see comment at `usePdfPreview.ts:70`. This avoids blank-screen flashes during transient render failures.

Callouts

ID	Label	Citation	Confidence
F-1	FacetAiProxyError class	src/utils/aiProxyErrors.ts:31	high
F-2	FacetApiError class	src/utils/facetApiErrors.ts:10	high
F-3	readAiProxyError categorizer	src/utils/aiProxyErrors.ts:106	high
F-4	rate-limit classification (429)	src/utils/aiProxyErrors.ts:177	high
F-5	overload classification (529 / 5xx temp)	src/utils/aiProxyErrors.ts:189	high
F-6	JsonExtractionError class	src/utils/llmProxy.ts:43	high

ID	Label	Citation	Confidence
F-7	callLlmProxy AbortError → timeout msg	src/utils/llmProxy.ts:282	high
F-8	JsonExtractionError kind='empty-sentinel'	src/utils/llmProxy.ts:147	high
F-9	JsonExtractionError kind='no-json-found'	src/utils/llmProxy.ts:154	high
F-10	prepGenerator JSON catch	src/utils/prepGenerator.ts:1931	high
F-11	coverLetterGenerator JSON catch	src/utils/coverLetterGenerator.ts:404	high
F-12	thesisGenerator wraps JsonExtractionError	src/utils/thesisGenerator.ts:650	high
F-13	jobMatch 2-attempt retry loop	src/utils/jobMatch.ts:1266	high
F-14	usePdfPreview.onerror — no retry	src/hooks/usePdfPreview.ts:40	high
F-15	typst.worker error frame	src/engine/typst.worker.ts:36	high
F-16	typst.worker missing-template guard	src/engine/typst.worker.ts:8	high
F-17	unknown templateId → fallback DEFAULT	src/hooks/usePdfPreview.ts:103	high
F-18	snapshot version mismatch throw	src/persistence/validation.ts:348	high
F-19	research_budget_exceeded handler	src/routes/research/ResearchPage.tsx:2506	high
F-20	ai_capability_unavailable handler	src/routes/research/ResearchPage.tsx:2517	high
F-21	toFacetApiError AbortError → 'offline'	src/utils/facetApiErrors.ts:89	high
F-22	toFacetApiError TypeError → 'offline'	src/utils/facetApiErrors.ts:96	high
F-23	resolvePersistenceFailurePhase	src/persistence/coordinator.ts:122	high
F-24	coordinator bootstrap catch	src/persistence/coordinator.ts:328	high
F-25	coordinator save catch	src/persistence/coordinator.ts:385	high
F-26	missing mergeImportedSnapshot	src/persistence/coordinator.ts:402	high
F-27	IndexedDB open failure	src/persistence/indexedDb.ts:48	high
F-28	IndexedDB transaction abort	src/persistence/indexedDb.ts:87	high
F-29	IndexedDB unavailable	src/persistence/indexedDb.ts:25	high
F-30	fallback to localStorage	src/persistence/indexedDb.ts:240	high
F-31	save AbortError → false	src/persistence/fileSystemAccess.ts:95	high
F-32	open AbortError → null		high

ID	Label	Citation	Confidence
		src/persistence/ fileSystemAccess.ts:123	
F-33	browser capability guard (showSaveFilePicker)	src/persistence/ fileSystemAccess.ts:82	high
F-34	backup parseEnvelope invalid JSON	src/persistence/ backupBundle.ts:96	high
F-35	AES-GCM decrypt failure	src/persistence/ backupBundle.ts:197	high
F-36	WebCrypto unavailable	src/persistence/ backupBundle.ts:35	high
F-37	normalizeWorkspaceSnapshot silent backfill	src/persistence/ normalization.ts:89	high
F-38	exhaustive artifact type check	src/persistence/validation.ts:336	high
F-39	research activeThesisId referential integrity	src/persistence/validation.ts:317	high
F-40	cover letter conflict → console.warn skip	src/persistence/ workspaceImportMerge.ts:82	high
F-41	research poll error continues polling	src/routes/research/ ResearchPage.tsx:1389	high
F-42	getResearchJobPollDelay backoff	src/utils/deepSearchClient.ts:57	high
F-43	streamDeepResearchJob abort suppression	src/utils/deepSearchClient.ts:167	high
F-44	WorkspaceBackupDialog catch-all	src/components/ WorkspaceBackupDialog.tsx:175	high
F-45	PresetSaveCanceledError class	src/hooks/usePresets.ts:28	high
F-46	preset cancel suppressed	src/hooks/usePresets.ts:221	high
F-47	importResumeConfig parse wrap	src/engine/serializer.ts:813	high
F-48	jdParser never throws — silent degradation	src/utils/jdParser.ts:26	high
F-49	PipelinePage per-entry analysis errors	src/routes/pipeline/ PipelinePage.tsx:231	high
F-50	BuildPage density optimization failure	src/routes/build/ BuildPage.tsx:1003	high

Narrative

LLM proxy error taxonomy — [F-1] FacetAiProxyError

[F-3] readAiProxyError is the central classifier. It examines the HTTP response and the error message body, producing a FacetAiProxyError with one of: ai_rate_limited (429 or message-match), ai_overloaded (529 or 5xx + temporary-capacity message), research_budget_exceeded,

ai_capability_unavailable. Two routes have specific handlers: [F-19] for budget exceeded, [F-20] for capability unavailable (with reason='opus_unavailable' as a sub-discriminator). Other generators just bubble the error to the user.

[F-7] is the AbortError-to-timeout transformation in callLlmProxy. The 30s default is in llmProxy.ts:209; searchExecutor uses 120s.

JSON parse failure handling

[F-6] JsonExtractionError carries a discriminator kind: - [F-8] empty-sentinel: <result></result> tags present but body empty - [F-9] no-json-found: no sentinel, no fenced block, no balanced braces

Three generators have specific handlers: [F-10] prepGenerator, [F-11] coverLetterGenerator, [F-12] thesisGenerator. They distinguish the typed parse error (re-throw) from generic parse errors (wrap with a user-facing message). thesisGenerator additionally wraps with a friendlier message: “Generated search thesis response was malformed. Try regenerating the thesis.”

Retry: where it does and doesn't exist

Retry exists in two places: - [F-13] jobMatch's 2-attempt loop wrapping callLlmProxy - [F-41] research poll loop continues on transient errors with [F-42] stepped backoff

Retry does NOT exist for: - Typst worker render failures ([F-14] confirmed by absence-claim grep — no retry/backoff in src/engine/ or src/hooks/usePdfPreview.ts) - callLlmProxy itself (transient errors throw to caller; only jobMatch wraps) - IndexedDB transaction failures (rejection bubbles up)

The Typst-no-retry choice is mitigated by [F-30] (preserve old preview on error) and [F-17] (fall back to DEFAULT_TEMPLATE_ID on unknown template). Render correctness is preserved differently — by not blanking — rather than by retrying.

Persistence error binarization

[F-21] AbortError → FacetApiError code='offline'. [F-22] TypeError (network failure) → same. Everything in the persistence path that fails routes through [F-23] resolvePersistenceFailurePhase to either phase='offline' or phase='error'. UI consumers can show different messaging for each.

[F-24] and [F-25] are the catches in coordinator: bootstrap and save. Both set the failure phase, record lastError, and re-throw. The state machine update is the side effect; the throw is what the caller sees.

[F-29] IndexedDB-unavailable + [F-30] localStorage fallback: this means even a hostile browser environment that blocks IndexedDB still gets a working app with smaller persistence capacity.

Silent degradations (where the system chooses to swallow failure)

[F-37] `normalizeWorkspaceSnapshot` silently backfills missing artifacts (`linkedin/recruiter/debrief/jdAnalysis`) with empty payloads. This handles the schema-evolution case where an older snapshot lacks newer artifact types.

[F-40] `workspaceImportMerge` skips a cover letter if the local pipeline entry already has one — `console.warn` only, no error. Import doesn't fail the user's local state.

[F-48] `jdParser` **never throws**. It returns empty strings for unparseable fields. JD parsing is best-effort; the user can edit fields after.

[F-49] `PipelinePage` stores JD analysis errors per-entry in a map rather than as a page-level error. One bad analysis doesn't break the whole pipeline view.

[F-50] `BuildPage` density optimization failures show a notice but don't propagate. Density optimization is opportunistic; if the LLM call fails, the resume is unchanged.

File System Access — user cancel is not a failure

[F-31] and [F-32] distinguish `AbortError` (user pressed Cancel in the file picker) from real errors. `AbortError` returns `false/null`; real errors re-throw. [F-33] is the browser capability guard — Safari historically lacked these APIs.

Backup integrity

[F-34] invalid JSON → 'Backup file is not valid JSON.'. [F-35] AES-GCM decrypt failure → 'Passphrase is incorrect or the backup file is corrupted.' (single message; doesn't distinguish wrong passphrase from tampering — appropriate for end users). [F-36] `WebCrypto` missing → hard fail.

[F-26] is a contract guard, not a failure path

`importWorkspaceSnapshot` with `mode='merge'` throws if `mergeImportedSnapshot` isn't configured. This is a developer error guard, not a runtime failure mode — but it's worth keeping in the diagram because misconfiguring the runtime can manifest at import time.

Verification log

Discarded findings

None — every error class, catch site, and retry citation resolved at the claimed line.

Synthesized cap

- Synthesized share: $0/50 = 0\%$. All failure paths cite specific lines.

Absence claims

- [F-14] “no retry on Typst worker render failure” — subagent provided grep evidence, orchestrator independently re-grepped `retry|backoff` across `src/engine/` and `src/hooks/usePdfPreview.ts` and confirmed empty. **Absence claim survives.**

Unverified citations

- Specific line offsets within long files (e.g., `coverLetterGenerator.ts:404`, `jobMatch.ts:1266` was sample-verified; `thesisGenerator.ts:650`, `prepGenerator.ts:1931` trusted from subagent evidence). Catch sites in long generator files were not all individually deep-read; `jobMatch`’s retry loop was sample-verified at line 1266.

Open questions

- The asymmetry between [F-13] `jobMatch` having retry and other LLM callers not having retry — is this principled (some calls are idempotent, others aren’t?) or accidental? Worth a deliberate review.
- [F-30] “preserve old preview on render error” depends on the previous preview not being garbage-collected. If the user navigates away and back, the cached blob URL is gone — first-render-after-navigation has no fallback. Is that bug or feature?
- [F-37] silent backfill of missing artifacts handles schema *additions* gracefully; what about schema *renames* or field-type changes? Need to inspect `validation.ts` migration logic.
- The deep-research [F-41] keeps polling on error indefinitely (with [F-42] capped at 30s intervals). Is there a max-attempts circuit breaker, or does the user have to abort manually?